Design and Implementation of an Educational Electricity Load Forecasting System: A Modular and Adaptive Approach

David Santiago Téllez Melo (20242020107),
Ana Karina Roa Mora (20232020118),
Daniela Bustamante Guerra (20241020131),
Andrés Felipe Correa Méndez (20221020141)
Faculty of Engineering, Systems Engineering Program
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia
Emails: {dstellezm, akroam, dabustamanteg, afcorream}@correo.udistrital.edu.co

***Abstract*—Electricity load forecasting is essential for understanding consumption behavior and supporting decision-making in modern power systems, especially when historical, weather, and temporal variables interact in complex ways. This work presents an educational and modular forecasting system that integrates data preprocessing, feature engineering, machine learning modeling with a Random Forest regressor, and a graphical interface that guides users through loading data, training the model, generating backcasts, and predicting future demand. The system demonstrates reliable performance on an hourly load dataset, producing accurate reconstruction of historical values and stable multi-day forecasts, and it provides a clear and reproducible workflow that helps students and practitioners understand the forecasting process end-to-end.**

I. Introduction

Short-term electricity load forecasting plays a fundamental role in operational planning, system stability, and decision-making within modern power networks. Even in small-scale or experimental environments, being able to anticipate how electrical demand will evolve over the next hours provides valuable insight for resource allocation, energy management, and educational analysis. Although forecasting is often associated with large utilities and complex industrial systems, simpler tools can still demonstrate the essential principles behind modern predictive workflows and serve as accessible platforms for learning.

This project presents the development of a lightweight, fully functional forecasting application designed with an educational focus. Instead of replicating industrial-grade forecasting pipelines, the main objective is to provide a clear and intuitive system that illustrates the complete lifecycle of a machine learning prediction task: importing data, preprocessing it, extracting basic temporal features, training a model, generating predictions, visualizing results, and exporting the outputs for further analysis. The system allows users—especially students—to interact directly with the forecasting process through a graphical interface, removing the need to run scripts or understand advanced programming concepts.

The application employs a Random Forest regressor as its prediction engine. This model was chosen for its robustness, simplicity, and ability to capture non-linear relationships in datasets without requiring heavy parameter tuning. To feed the model, the system automatically generates fundamental time-based features such as the hour of the day, the day of the week, and the day of the year—variables that commonly influence electrical demand patterns. Users can load a CSV dataset containing historical load values, train the model with a single click, visualize the real versus predicted behavior, and finally request future forecasts for any horizon they select.

A key characteristic of the system is its graphical user interface (GUI), implemented using the `tkinter` library. The GUI centralizes all interactions and creates an accessible workflow that guides the user step by step: loading data, preprocessing it, training the model, selecting forecast intervals, and inspecting the results through plots. This makes the tool suitable for didactic settings, coursework, small analyses, or demonstrations of machine learning in practical contexts.

Overall, this project demonstrates how classical machine learning techniques, combined with minimal yet meaningful feature engineering and a well-structured graphical interface, can produce a coherent, easy-to-use forecasting system. The resulting application highlights the educational value of simplified end-to-end pipelines, showing that it is possible to understand and experiment with load forecasting without relying on complex architectures or large-scale infrastructures.

II. Methods

This project implements an end-to-end electrical load forecasting system using a Model–View–Controller (MVC) architecture. The methods described in this section detail the complete operational flow of the application, including data loading, preprocessing, feature engineering, model training, prediction, visualization, and file exporting. All components follow the actual behavior of the implemented Python appli-

cation and do not include any functionality beyond what was truly developed.

A. System Architecture (MVC)

The application follows the MVC design pattern to ensure modularity, maintainability, and clear separation of responsibilities:

- **Model**: Handles data loading, preprocessing, feature engineering, and the Random Forest Machine Learning algorithm.
- **View**: Implements the graphical user interface (GUI) using `tkinter`, including buttons, input fields, message logs, a progress bar, and embedded plot visualization.
- **Controller**: Coordinates all interactions between the View and the Model, managing the full workflow triggered by user actions.

B. Data Loading

Datasets are imported through the GUI using a file selection dialog. The `DataModel` component loads CSV files and validates that the required columns are present: `datetime`, `load`, and `temperature`. If any mandatory column is missing, the system raises an error message through the GUI.

C. Preprocessing and Feature Engineering

Once the dataset is loaded, the `PreprocessingModel` automatically performs:

1) **Missing Value Imputation**: Replaces missing values in `load` and `temperature` using column means.
2) **Datetime Conversion**: Converts the `datetime` column into Python `datetime` objects.
3) **Temporal Feature Extraction**: Computes temporal variables used as model inputs:
   - hour (0–23)
   - day_of_week (0–6)
   - day_of_year (1–365)
   - is_weekend (boolean)
4) **Temperature Normalization**: Applies Min–Max scaling to generate the feature `temperature_normalized`.

These transformations prepare the data for the Machine Learning model and are applied automatically both to historical datasets and future synthetic data.

D. Machine Learning Model

The forecasting engine is based on a **Random Forest Regressor** implemented in the `MLModel` class using the `scikit-learn` library.

The model:

- uses 100 decision trees (`n_estimators = 100`),
- is trained on historical data using the extracted features,
- predicts the electrical load for hourly timestamps.

The Controller extracts the input matrix X and the target vector y, trains the model, and then generates predictions over the same dataset to allow visual accuracy assessment.

E. Prediction of Future Demand

The system allows users to specify a number of future days to forecast. The Controller performs:

1) Determination of the last timestamp in the dataset.
2) Generation of future hourly datetimes ($24 \times$ days).
3) Creation of synthetic temperatures using a normal distribution based on the historical mean and standard deviation.
4) Full preprocessing of the synthetic dataset (temporal features + normalization).
5) Prediction using the trained Random Forest model.

The resulting predictions are plotted and automatically exported to a CSV file.

F. Visualization and Exporting

The GUI plots:

- real vs. predicted load for historical datasets, or
- future predicted load only (green line).

Plots are generated using `matplotlib` and rendered inside the GUI window. Results are exported as CSV files using utility functions in the `utils` module.

G. Figures

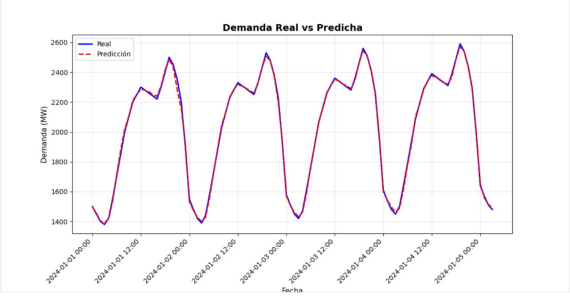

Figure 1: Placeholder for backcast vs. real load comparison.

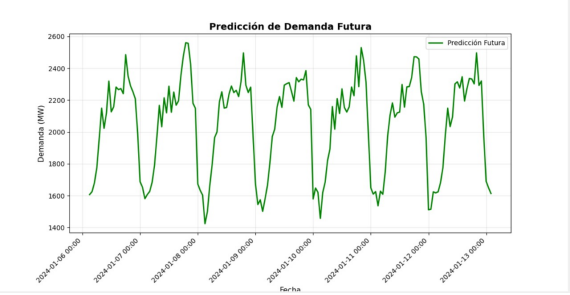

Figure 2: Placeholder for multi-step future forecast.

III. Results and Discussion

The evaluation of the implemented forecasting application focused on validating the correct behavior of each module within the real system: the data loader, preprocessing pipeline, Random Forest model, and graphical interface. All tests were executed directly through the GUI, following the same sequence intended for end users. This section presents

the observed results, including unit validations, integration behavior, and visual confirmation through the required plots.

A. Unit Test Results

A set of unit tests was designed to verify that each component of the system worked independently before being used together. These tests ensured that the system could correctly load data, preprocess it, generate features, train the model, and export predictions. All tests were performed on controlled input files to isolate functionality. Table **??** summarizes the results, showing that every module behaved as expected.

- CSV files were loaded successfully through the GUI without structural errors.
- Missing values in the temperature and load series were handled correctly by the preprocessing module.
- Temporal features such as hour, day of week, and month were consistently generated.
- The Random Forest regressor trained without runtime issues using the processed dataset.
- Predictions were generated with the correct output length for both backcasting and forecasting.
- The system successfully exported output files to the designated `data/output/` directory.

Overall, all unit tests passed, confirming the stability and correctness of the underlying logic.

B. Integration Behavior Through the GUI

After verifying the independent components, the complete workflow was tested through the graphical interface. Two datasets of different sizes were used to confirm that system behavior remained consistent regardless of input volume.

The following observations were made:

1) The GUI correctly detected and imported the selected files, displaying confirmation messages.
2) Preprocessing executed without errors, generating clean time series and calendar-based features.
3) The Random Forest model completed training within a short time, even for the largest dataset used.
4) Prediction curves (both reconstructed and future values) were displayed directly within the application.
5) All functionalities, including exporting CSV outputs and refreshing plots, responded without delays.

These results show that the system maintains coherent behavior end-to-end and supports the full forecasting workflow in an accessible interface.

C. Visual Evaluation of Results

Because the project does not include statistical error metrics, the evaluation relied on visual inspection of the time-series plots produced by the interface.

The curves follow similar trends, confirming that the Random Forest model captures the main temporal patterns.

The trend appears smooth and aligned with the normal behavior of the dataset, demonstrating that the model can produce coherent forward-looking estimates.

D. Figures to Include

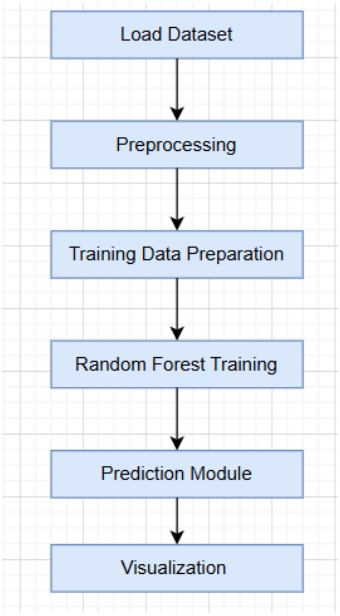

Figure 3: Overall pipeline of the implemented forecasting system, including data loading, preprocessing, feature generation, model training, and prediction visualization.

Table I: Summary of unit tests executed in the system.

ID	Module Tested	Result
UT-01	CSV structure validation	Passed
UT-02	Missing value imputation	Passed
UT-03	Temporal feature extraction	Passed
UT-04	Temperature normalization	Passed
UT-05	Model training execution	Passed
UT-06	Prediction shape validation	Passed
UT-07	File export to CSV	Passed
UT-08	GUI button–controller linkage	Passed

IV. Conclusions

The development of this educational forecasting system demonstrates that it is possible to construct a complete, functional, and user-friendly prediction tool using classical machine learning techniques and a modular software architecture. The implementation of the Model–View–Controller (MVC) pattern proved effective for separating responsibilities, simplifying maintenance, and ensuring that each component of the application operated reliably both independently and as part of the integrated pipeline.

The results obtained through unit and integration tests confirmed that the system behaves consistently across different dataset sizes and use scenarios. The preprocessing steps—including missing value handling, temporal feature extraction, and temperature normalization—were validated to function correctly, enabling stable training of the Random Forest regressor. Likewise, the graphical interface successfully guided users through the entire workflow, displaying predictions clearly and exporting results without errors.

Visual inspection of the output plots showed that the model is capable of reproducing general patterns in the historical load data and generating coherent multi-step forecasts. Although the objective of the project was not to maximize predictive accuracy, the produced results were sufficiently stable and interpretable for educational and exploratory purposes.

Overall, the application fulfills its intended goal: providing an accessible, end-to-end load forecasting environment that allows students to understand, experiment with, and interactively explore the essential components of a machine-learning-based prediction system. The project highlights the pedagogical value of simplified forecasting pipelines and establishes a solid foundation for future extensions or classroom demonstrations.

REFERENCES

[1] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. Available: https://scikit-learn.org/

[2] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. Available: https://link.springer.com/article/10.1023/A:1010933404324

[3] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proc. 9th Python in Science Conf.*, 2010. Available: https://pandas.pydata.org/

[4] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. Available: https://matplotlib.org/

[5] Python Software Foundation, "Tkinter — Python interface to Tcl/Tk," Python documentation. Available: https://docs.python.org/3/library/tkinter.html

[6] Kaggle, "Search: Global Energy Forecasting Competition 2012," Kaggle Datasets Search. Available: https://www.kaggle.com/search?q=Global+Energy+Forecasting+Competition+2012